

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 09-2

NOMBRE COMPLETO: Tapia Verastegui Venancio Miguel

Nº de Cuenta: 320324187

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 06

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 01/11/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA

Animación compleja de holocrón y cohete

1 Ejecución de los ejercicios

Se implementaron dos animaciones automáticas en C++ y OpenGL: las ocho esquinas del holocrón giran alrededor de su propio eje diagonal y un cohete despega para seguir una trayectoria curva calculada mediante una función Bézier. Se reutilizaron la cámara, iluminación, carga de modelos y shaders de las prácticas anteriores.

Rotación local del holocrón

Se recuperaron el cuerpo y las ocho esquinas independientes de la práctica 05. El cuerpo permanece fijo. Para cada esquina se conserva su centro p y se calcula el eje normalizado desde el centro del holocrón hasta p. El ángulo aumenta a 60 grados por segundo y se reduce por módulo de 360 grados.

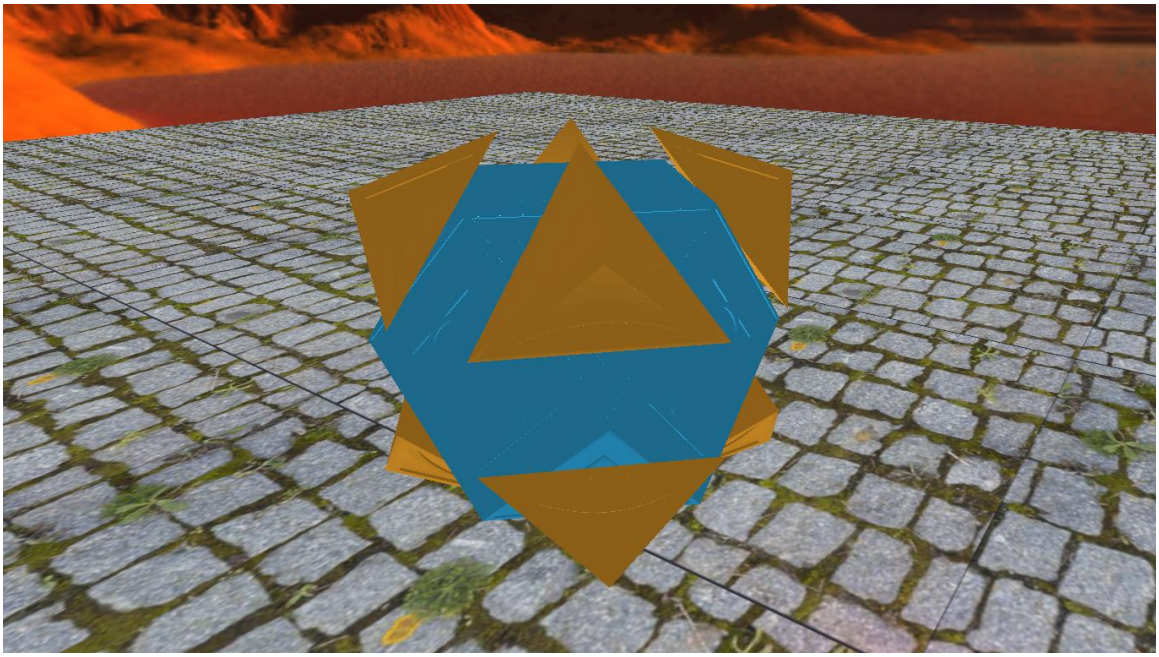


Figura 1. Holocrón con las ocho esquinas animadas sobre sus propios ejes.

La transformación se compone como M padre × T(p) × R(eje, ángulo) × T(−p). La traslación inversa lleva el pivote al origen; después se aplica la rotación y se devuelve la pieza a su posición. Esto mantiene fijo el centro de cada esquina durante el giro.

Código de rotación del holocrón

El siguiente bloque corresponde al archivo P09-2-320324187.cpp incluido en la entrega. El ciclo recorre las ocho piezas y aplica traslación al pivote, rotación sobre la diagonal local y traslación inversa. La matriz padre coloca el conjunto en la escena.

```
131  auto base=glm::translate(glm::mat4(1),posicionSatelite);
132  base=glm::scale(base,glm::vec3(2));base=glm::translate(base,-centro);
133  glUniform3f(uc,.2f,.75f,1);glUniformMatrix4fv(um,1,GL_FALSE,glm::value_ptr(base));Holoc
    ron.RenderModel();
134  for(int i=0;i<8;++i){
135  // T(p) R T(-p) mantiene fijo el centro de cada esquina.
136  auto m=glm::translate(base,pivotes[i]);m=glm::rotate(m,anguloSatelite[0],glm::normalize
    (pivotes[i]-centro));m=glm::translate(m,-pivotes[i]);
137  glUniform3f(uc,1,.68f,.18f);glUniformMatrix4fv(um,1,GL_FALSE,glm::value_ptr(m));Esquina
    s[i].RenderModel();}
```

Figura 2. Captura del código fuente de las transformaciones locales.

El ángulo compartido cambia continuamente con el tiempo; cada pieza utiliza su propio eje y pivote. Por ello los centros permanecen fijos aunque las esquinas cambien de orientación. No se requiere presionar teclas para iniciar la animación.

Trayectoria del cohete espacial

Se recuperó el modelo Saturn V.glb de Practica07/Previo07-insumos, seleccionado previamente para el previo 07. El modelo está acreditado a NASA y Michael D. Carbajal. Se convirtió a OBJ para el cargador existente, conservando geometría, coordenadas UV y texturas originales.

La trayectoria utiliza cuatro puntos de control: P0 = (20, 2, 0), P1 = (20, 48, 0), P2 = (30, 85, −20) y P3 = (110, 100, −45). La función Bézier cúbica combina los cuatro puntos mediante pesos polinomiales, como se observa en la captura de código. El comienzo es vertical y la trayectoria se curva progresivamente hacia un lado.

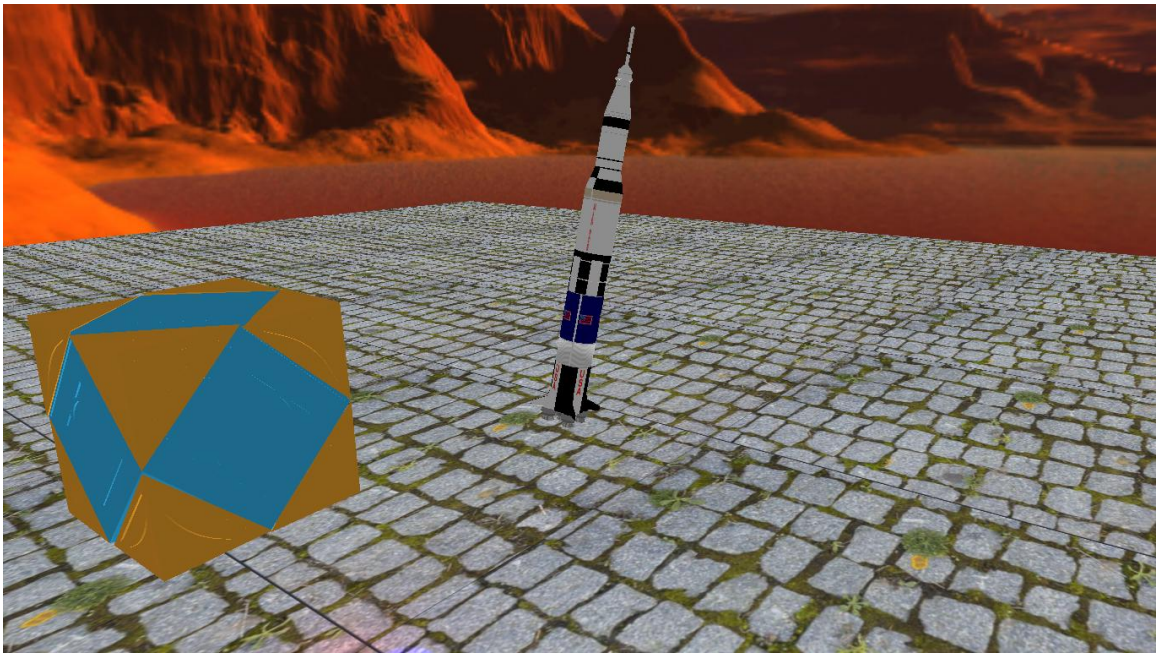


Figura 3. Cohete en ascenso y orientado según la tangente de su trayectoria.

Cada ciclo dura 24 segundos: espera inicial de 2 segundos, vuelo de 20 segundos y espera final de 2 segundos. Se usa u = s², con s normalizado entre cero y uno, para aumentar gradualmente la rapidez del despegue. El reinicio al punto inicial permite repetir la demostración; no representa un aterrizaje.

Comentarios generales y controles

1: vista general; 2: seguimiento del cohete; 3: holocrón; 0: cámara libre. En cámara libre se utilizan WASD, Q/E y mouse. Esc cierra la aplicación. Se incluyen código, proyecto, ejecutable y recursos. Fuente del cohete: NASA y Michael D. Carbajal, Saturn V, https://science.nasa.gov/3d-resources/saturn-v/.

Código de trayectoria y actualización temporal

El bloque siguiente muestra la curva Bézier, su tangente y el avance del tiempo. El parámetro normalizado se eleva al cuadrado para iniciar el vuelo suavemente; la tangente proporciona la dirección de la nariz del cohete.

```
111 glm::vec3 Trayectoria(float u){
112  float v=1-u;return v*v*v*glm::vec3(20,2,0)+3*v*v*u*glm::vec3(20,48,0)+3*v*u*u*glm::vec3
    (30,85,-20)+u*u*u*glm::vec3(110,100,-45);
113 }
114 glm::vec3 Tangente(float u){
115  float v=1-u;return
    3*v*v*glm::vec3(0,46,0)+6*v*u*glm::vec3(10,37,-20)+3*u*u*glm::vec3(80,15,-25);
116 }
117 void ActualizarAnimacion(float dt){
118  tiempoAnimacion+=dt;
119  float ciclo=std::fmod(tiempoAnimacion,24.f);
120  float u=glm::clamp((ciclo-2)/20,0.f,1.f);u=u*u; // aceleracion de despegue
121  posicionNave=Trayectoria(u);direccionNave=glm::normalize(Tangente(u));
122  anguloSatelite[0]=std::fmod(tiempoAnimacion*PI/3,2*PI);
123 }
```

Figura 4. Captura del código fuente de trayectoria y actualización automática.

Orientación y comprobaciones

La derivada de la curva define la dirección de vuelo. El eje Y local del cohete se alinea con esa tangente y los otros ejes se obtienen con productos cruzados. De este modo la nariz acompaña la curva en vez de conservar una orientación vertical durante todo el recorrido.

2 Problemas presentados y soluciones

Los OBJ del holocrón conservan coordenadas desplazadas del origen. Se mantuvieron los pivotes del proyecto original y se aplicó una matriz padre para colocar el conjunto. En el cohete se extrajeron las imágenes del GLB y se enlazaron mediante materiales OBJ. El modelo se centró horizontalmente, con la base en Y = 0, y se escaló a una altura de 24 unidades sin alterar sus proporciones.

Verificación realizada

Se compiló la aplicación en Release x64 con Visual Studio. La verificación numérica evalúa 1201 muestras de la curva y comprueba posiciones finitas, ascenso continuo y tangente no nula. Se generaron capturas mediante el modo --evidencia para revisar las dos animaciones. El movimiento se actualiza con deltaTime en segundos.

3 Conclusión

Esta práctica me pareció muy similar a la anterior, porque también trabajé con modelos que se mueven automáticamente y siguen una trayectoria. En esta ocasión cambiaron los movimientos: las esquinas del holocrón giran sobre su propio eje y el cohete despega siguiendo una curva. Me sirvió para repasar el orden de las transformaciones y entender mejor cómo orientar un modelo según su recorrido.

Complejidad y explicación

La dificultad principal fue combinar dos tipos de movimiento: rotaciones locales independientes para el holocrón y desplazamiento con orientación para el Saturn V. La trayectoria se evalúa directamente a partir del tiempo; la rotación de cada esquina utiliza un pivote fijo. La separación de estas funciones permite modificar la rapidez sin reconstruir la geometría.

Comentarios generales

En general sentí que esta práctica fue muy parecida a la anterior, así que ya tenía una idea de cómo hacer las animaciones. Lo que más tuve que revisar fue que las esquinas giraran en su lugar y que el cohete se inclinara conforme avanzaba. También reutilicé los modelos que ya tenía de otras prácticas. Ver el resultado en movimiento me ayudó a identificar los ajustes que hacían falta.

Bibliografía en formato APA

National Aeronautics and Space Administration. (2025, 14 de abril). Saturn V [Modelo 3D de NASA y Michael D. Carbajal]. NASA Science. https://science.nasa.gov/3d-resources/saturn-v/